

# INFORME TÉCNICO: DESPLIEGUE DE KUBERNETES CON LOAD BALANCER Y AWS

**Entorno:** Híbrido (WSL2 Ubuntu 24.04 LTS y Amazon Web Services)

**Tecnologías Clave:** Kubernetes (k3s), Docker, Flask, Túneles SSH

**Preparado por:** Enrique Crespo Ramírez **Fecha de finalización:** 14 de enero de 2026



# ÍNDICE

## 1. **Introducción**

- a. 1.1 Contexto del sistema híbrido
- b. 1.2 Objetivos del proyecto

## 2. **Preparación del Entorno Local (WSL2)**

- a. 2.1 Verificación del sistema y kernel de Microsoft
- b. 2.2 Instalación de k3s (Kubernetes Ligero)
- c. 2.3 Configuración de kubectl y permisos de acceso

## 3. **Creación de la Aplicación Web (Flask)**

- a. 3.1 Estructura del proyecto y frontend
- b. 3.2 Desarrollo de la lógica de red y endpoints de salud
- c. 3.3 Gestión de dependencias y Dockerfile

## 4. **Configuración de Kubernetes (Manifiestos YAML)**

- a. 4.1 Definición y aislamiento mediante Namespaces
- b. 4.2 Despliegue con réplicas y sondas de salud (Probes)

- c. 4.3 Integración con Docker Desktop e importación de imágenes

- d. 4.4 Exposición del servicio de tipo Load Balancer

## **5. Verificación del Balanceo de Carga Local**

- a. 5.1 Acceso mediante Port-Forward

- b. 5.2 Pruebas de carga iterativas con Curl

## **6. Configuración en AWS**

- a. 6.1 Reglas de seguridad y Security Groups

- b. 6.2 Provisión de instancia EC2 (t3.micro)

- c. 6.3 Conexión remota SSH y despliegue de k3s en la nube

## **7. Conectividad Híbrida y Escalado**

- a. 7.1 Establecimiento del Túnel SSH Inverso

- b. 7.2 Verificación de balanceo desde la instancia de AWS

- c. 7.3 Escalado dinámico de réplicas

## **8. Monitoreo y Observabilidad**

- a. 8.1 Análisis de logs y estados de salud

- b. 8.2 Métricas de rendimiento de CPU y memoria

- c. 8.3 Auditoría de eventos del clúster

## **9. Finalización y Limpieza**

- a. 9.1 Eliminación de recursos en Kubernetes y AWS
- b. 9.2 Apagado de servicios locales

## **10. Conclusiones y Reflexión Final**

# 1. Introducción

Este documento detalla el proceso de configuración y despliegue de una arquitectura de microservicios utilizando **Kubernetes (k3s)** sobre un entorno híbrido compuesto por **WSL2 (Ubuntu 24.04 LTS)** y **Amazon Web Services (AWS)**. El objetivo principal es demostrar la capacidad de orquestación de contenedores, balanceo de carga, escalado dinámico y conectividad segura mediante túneles SSH.

## 2. Objetivos

- Configurar un entorno de Kubernetes ligero (k3s) en un sistema local.
- Desplegar una aplicación web basada en Flask con múltiples réplicas.
- Implementar un servicio de tipo Load Balancer para la distribución de tráfico.
- Establecer una conexión segura entre el clúster local y una instancia EC2 en la nube de AWS.
- Monitorizar recursos y realizar escalado horizontal bajo demanda.

# PARTE 0: PREPARACIÓN DEL ENTORNO LOCAL (WSL2)

## 0.1 Contexto del Sistema y Usuario

Se inicia el entorno utilizando WSL2 con la distribución Ubuntu 24.04 LTS. Se verifica la identidad del usuario y la versión del kernel para asegurar la compatibilidad con las funciones de red de Kubernetes.

```
alumno09@A6Alumno09:/mnt/c/Windows/System32$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
alumno09@A6Alumno09:/mnt/c/Windows/System32$ whoami
alumno09
alumno09@A6Alumno09:/mnt/c/Windows/System32$ uname -r
6.6.87.2-microsoft-standard-WSL2
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

*Descripción: Verificación del sistema operativo Ubuntu 24.04, usuario 'alumno09' y versión del kernel de Microsoft-WSL2.*

## 0.2 Instalación de k3s (Kubernetes Ligero)

Se procede a la instalación de **k3s**, una distribución certificada de Kubernetes optimizada para entornos con recursos limitados. Se configura el modo de permisos del fichero kubeconfig a 644 para facilitar la administración.

```
[INFO] Creating k3s install script /usr/local/bin/k3s-install.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s
alumno09@A6Alumno09:/mnt/c/Windows/System32$ sudo k3s --version
k3s version v1.34.3+k3s1 (48ffa7b6)
go version go1.24.11
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

*Descripción: Ejecución del script de instalación de k3s y confirmación de la creación de servicios systemd. Se verifica que la versión instalada sea la correcta y que el servicio esté operativo.*

### 0.3 Instalación y Configuración de kubectl

Para la administración profesional del clúster, se instala el cliente oficial `kubectl`. Se configura el acceso al clúster copiando el fichero de configuración al directorio del usuario (`$HOME/.kube/config`) para evitar el uso constante de `sudo`.

```

C:\Users\h3ll0x>cmd /c curl -LO "https://dl.k8s.io/release/5/curl -L -s https://dl.k8s.io/release/stable.txt|bin/linux/amd64/kubectl"
kudo chmd > kubectl
kudo > kubectl top/pod/total/total/
% Total % Received % Xferd Average Speed Time Time Time Current
                                Dload Upload   0.00 0.00   0.00 0.00
100 138 100 138 0 52.69 0 0 0:00:01 0:00:01 0:00:01 74.5M
100 55.0M 100 55.0M 0 52.69 0 0 0:00:01 0:00:01 0:00:01 74.5M
C:\Users\h3ll0x>cmd /c cd\windows\system32 kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.3

```

*Descripción: Descarga del binario kubectl y asignación de permisos de ejecución.*

```
Kustomize Version: v5.7.1
alumno09@A6Alumno09:/mnt/c/Windows/System32$ mkdir -p $HOME/.kube
sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
chmod 600 $HOME/.kube/config
alumno09@A6Alumno09:/mnt/c/Windows/System32$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
a6alumno09    Ready    control-plane   21m   v1.34.3+k3s1
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

*Descripción: Verificación del nodo local en estado 'Ready' mediante el comando 'kubectl get nodes'.*

# PARTE 1: CREACIÓN DE LA APLICACIÓN WEB (FLASK)

## 1.1 Estructura del Proyecto

Se organiza el proyecto siguiendo los estándares de desarrollo: un directorio raíz para la aplicación, una carpeta templates para el frontend y ficheros de dependencias.

```
a6alumno09 Ready control-plane 21m v1.34.3+k3s1
alumno09@A6Alumno09:/mnt/c/Windows/System32$ cd ~
mkdir flask-app
cd flask-app
mkdir templates
touch app.py requirements.txt templates/index.html
alumno09@A6Alumno09:~/flask-app$ ls -R
.:
app.py requirements.txt templates

./templates:
index.html
alumno09@A6Alumno09:~/flask-app$
```

Descripción: Creación de la estructura de directorios 'flask-app' y 'templates'.

## 1.2 Desarrollo de la Lógica (app.py)

Se desarrolla una aplicación en Python con Flask que expone endpoints para información del Pod y comprobaciones de salud (/health). La aplicación está diseñada para identificar dinámicamente el nombre del Pod que responde, permitiendo visualizar el balanceo de carga.



```
alumno09@A6Alumno09: ~/flask-app
GNU nano 7.2
cat > app.py << 'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys

app = Flask(__name__)

# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
        'timestamp': datetime.now().isoformat()
    })

@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF

sudo chmod +x app.py
```

*Descripción: Código fuente de app.py con los endpoints de salud e información del sistema.*

## 1.3 Interfaz de Usuario e index.html

Se crea una plantilla HTML moderna con estilos CSS embebidos para mostrar visualmente los datos del balanceador de carga.

```
background: white;
padding: 50px;
border-radius: 10px;
box-shadow: 0 10px 25px rgba(0,0,0,0.2);
text-align: center;
max-width: 500px;
}
hi {
  color: #667eea;
  margin: 0 0 30px 0;
}
.info {
  background: #fefef0;
  padding: 20px;
  border-radius: 5px;
  margin: 20px 0;
}
.pod-name {
  font-size: 28px;
  color: #764ba2;
  font-weight: bold;
  font-family: monospace;
}
.timestamp {
  color: #666;
  font-size: 13px;
  margin-top: 10px;
}
.instruction {
  background: #e0f2fe;
  padding: 15px;
  border-radius: 5px;
  color: #0069ad;
  font-size: 14px;
  margin-top: 20px;
}
.refresh-button {
  margin-top: 20px;
  padding: 10px 20px;
  background: #667eea;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 14px;
}
.refresh-button:hover {
  background: #764ba2;
}
</style>
</head>
<body>
<div class="container">
<h1>Kubernetes Load Balancer</h1>
<div class="info">
  <p style="margin: 0 0 10px 0; color: #666;">Pod atendiendo solicitud:</p>
  <p class="pod-name" id="pod-name">Cargando...</p>
  <p class="timestamp" id="timestamp"></p>
</div>
</body>
</html>
```

Descripción: Contenido del archivo index.html diseñado para mostrar el nombre del Pod atendiendo la solicitud.

## 1.4 Gestión de Dependencias y Containerización

Se define el archivo requirements.txt y se crea un **Dockerfile** optimizado basado en python:3.12-slim.

```
alumno09@A6Alumno09: ~/kubernetes-aws-practice/app
GNU nano 7.2
Flask==3.0.0
Werkzeug==3.0.0
```

Descripción: Archivo requirements.txt con las versiones de Flask y Werkzeug.

```
alumno09@A6Alumno09: ~/kubernetes-aws-practice/app
GNU nano 7.2
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY index.html .

EXPOSE 5000

CMD ["python", "app.py"]
```

*Descripción: Definición del Dockerfile para la construcción de la imagen ultraligera.*

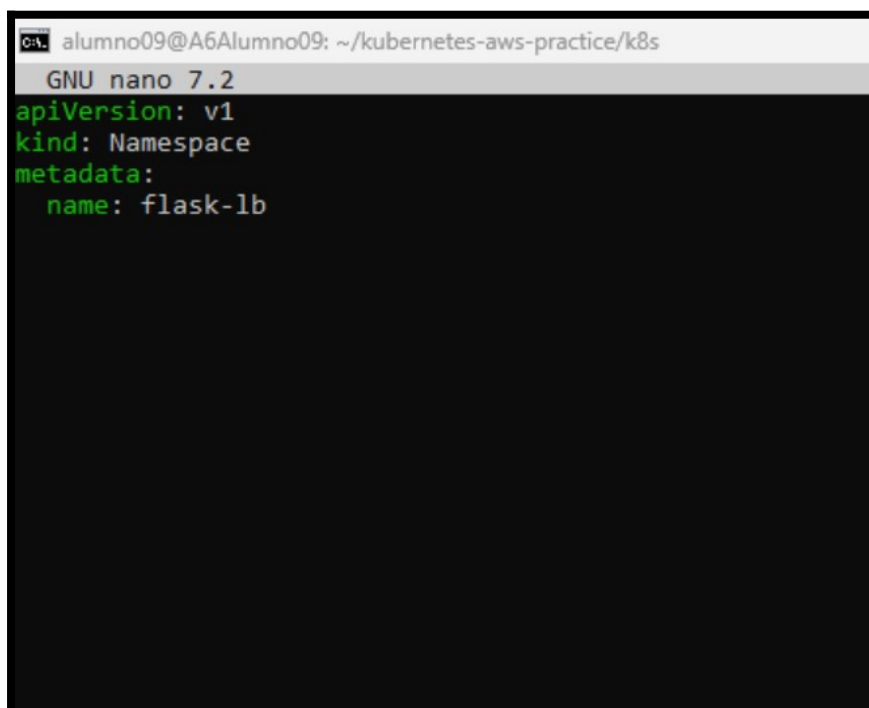
```
-s: command not found
alumno09@A6Alumno09:~/kubernetes-aws-practice/app$ ls -l
total 16
-rw-r--r-- 1 alumno09 alumno09 184 Jan 12 09:55 Dockerfile
-rw-r--r-- 1 alumno09 alumno09 899 Jan 12 09:47 app.py
-rw-r--r-- 1 alumno09 alumno09 2356 Jan 12 09:50 index.html
-rw-r--r-- 1 alumno09 alumno09 29 Jan 12 09:53 requirements.txt
alumno09@A6Alumno09:~/kubernetes-aws-practice/app$
```

*Descripción: Listado detallado de los archivos creados en el directorio de la aplicación.*

## PARTE 2: CONFIGURACIÓN DE KUBERNETES (MANIFESTS YAML)

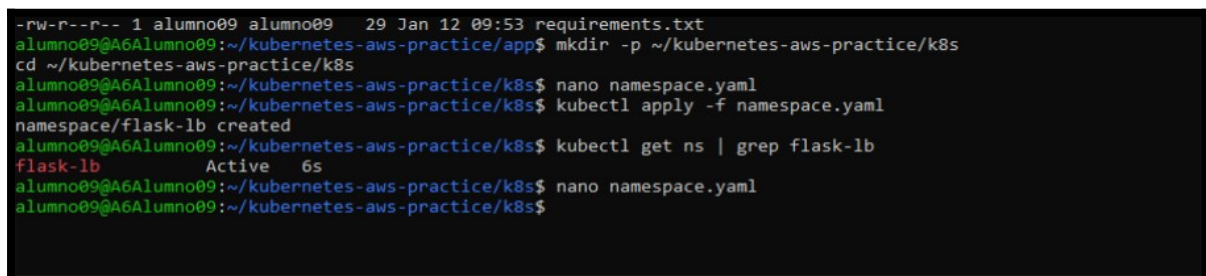
### 2.1 Aislamiento de Recursos (Namespace)

Se crea el Namespace `flask-lb` para organizar y aislar todos los recursos relacionados con esta práctica, siguiendo las mejores prácticas de administración de Kubernetes.



```
alumno09@A6Alumno09: ~/kubernetes-aws-practice/k8s
GNU nano 7.2
apiVersion: v1
kind: Namespace
metadata:
  name: flask-lb
```

*Descripción: Manifiesto YAML para la definición del Namespace 'flask-lb'.*



```
-rw-r--r-- 1 alumno09 alumno09 29 Jan 12 09:53 requirements.txt
alumno09@A6Alumno09:~/kubernetes-aws-practice/app$ mkdir -p ~/kubernetes-aws-practice/k8s
cd ~/kubernetes-aws-practice/k8s
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ nano namespace.yaml
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ kubectl apply -f namespace.yaml
namespace/flask-lb created
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ kubectl get ns | grep flask-lb
flask-lb          Active   6s
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ nano namespace.yaml
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$
```

*Descripción: Aplicación del Namespace y verificación de su estado 'Active'.*

## 2.2 Despliegue de la Aplicación (Deployment)

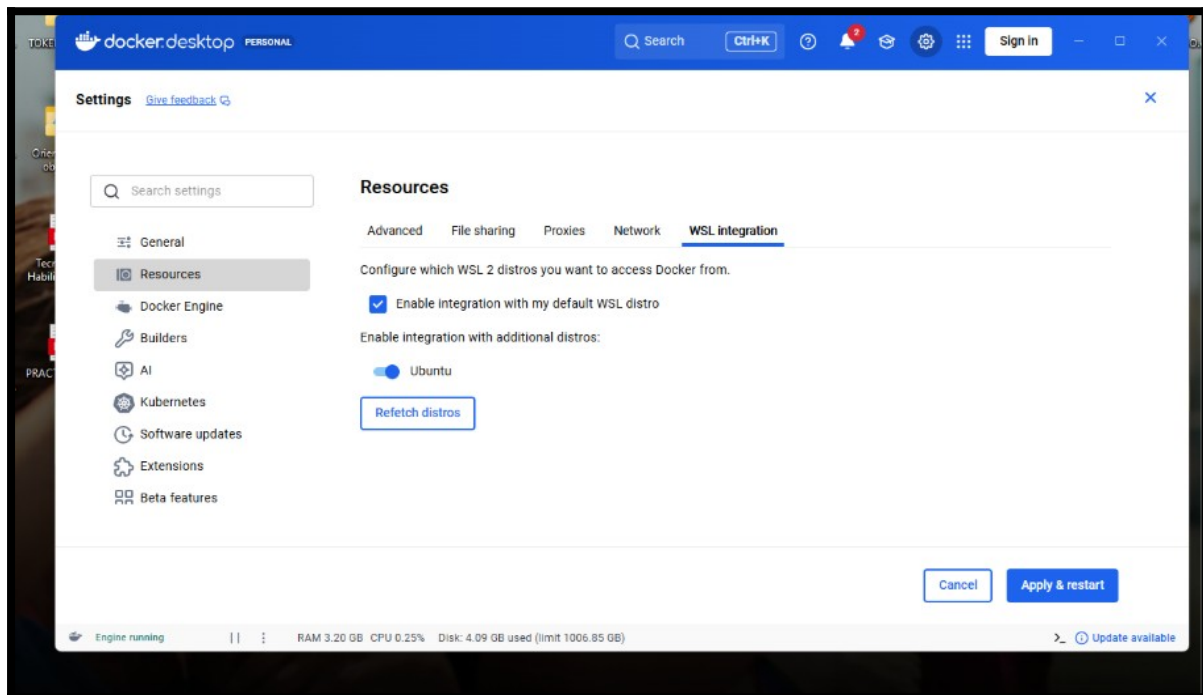
Se configura un Deployment con **3 réplicas**. Se incluyen sondas de salud (Liveness y Readiness) y se inyectan variables de entorno mediante fieldRef para que el contenedor conozca su propia identidad dentro del clúster.

```
alumno09@A6Alumno09: ~/kubernetes-aws-practice/k8s
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  namespace: flask-lb
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: flask-app:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: POD_NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 20
```

*Descripción: Manifiesto Deployment indicando 3 réplicas y configuración de Probes.*

## 2.3 Integración con Docker Desktop e Importación de Imagen

Para trabajar con imágenes locales, se habilita la integración de Docker Desktop con la distribución de Ubuntu en WSL2.



*Descripción: Ajustes de configuración en Docker Desktop para habilitar la integración con WSL2.*

```

C:\Windows\System32
Microsoft Windows [Versión 10.0.26100.5074]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>wsl --shutdown

C:\Windows\System32>
C:\Windows\System32>wsl -d Ubuntu
alumno09@A6Alumno09: /mnt/c/Windows/System32$
alumno09@A6Alumno09: /mnt/c/Windows/System32$ docker context ls
NAME                DESCRIPTION                DOCKER ENDPOINT                ERROR
default *           Current DOCKER_HOST based configuration   unix:///var/run/docker.sock
desktop-linux        Docker Desktop              npipe:///pipe/dockerDesktopLinuxEngine
CONTAINER ID        IMAGE                      COMMAND                        CREATED    STATUS    PORTS
73267b4d9b6b        mysql:5.7                  "/docker-entrypoint.s..."    6 weeks ago    Up 1 second    3306/tcp, 33060/tcp
4948f4e46684        phpmyadmin/phpmyadmin     "/docker-entrypoint.s..."    6 weeks ago    Up 1 second    0.0.0.0:8082->80/tcp, [
alumno09@A6Alumno09: /mnt/c/Windows/System32$

```

*Descripción: Verificación de la conexión entre el cliente Docker en Linux y el motor en Windows.*

Se construye la imagen y se importa manualmente al runtime de k3s (containerd) para que los nodos puedan instanciar los contenedores sin necesidad de un registro externo.



```

alumno09@A6Alumno09:/mnt/c/Windows/System32$ cd ~/kubernetes-aws-practice/app
docker build -t flask-app:1.0 .
[+] Building 8.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 223B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:a75662d0fec8d98bd7161c91050be2e0a9b21d284f3b7a7253d5db25f7d583fb3
=> => resolve docker.io/library/python:3.12-slim@sha256:a75662d0fec8d98bd7161c91050be2e0a9b21d284f3b7a7253d5db25f7d583fb3
=> => sha256:d7c2baa095fbf75d83cd43b216be2d72bef124518338e23288648addadb8ee18 250B / 250B
=> => sha256:61a9f733534e478a7570aaf3a154be5b9302dcf474e1fe0a827d4b146543b40e 12.11MB / 12.11MB
=> => sha256:fe2d526e7d41df5bda23740a00f6b254c44f67ce9bfc9507b36fb77c3b4c38e8 1.29MB / 1.29MB
=> => sha256:02d7611c4eae219af91448a4720bdba036575d3bc0356cfe12774af85daa6aff 29.78MB / 29.78MB
=> => extracting sha256:02d7611c4eae219af91448a4720bdba036575d3bc0356cfe12774af85daa6aff
=> => extracting sha256:fe2d526e7d41df5bda23740a00f6b254c44f67ce9bfc9507b36fb77c3b4c38e8
=> => extracting sha256:61a9f733534e478a7570aaf3a154be5b9302dcf474e1fe0a827d4b146543b40e
=> => extracting sha256:d7c2baa095fbf75d83cd43b216be2d72bef124518338e23288648addadb8ee18
=> [internal] load build context
=> => transferring context: 3.40kB
=> [2/6] WORKDIR /app
=> [3/6] COPY requirements.txt .
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> [5/6] COPY app.py .
=> [6/6] COPY index.html .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:a0e982f703ddc4d1743643e24349dc121daf7ebc4fd46d5ed18a4b239cfe0e2
=> => exporting config sha256:2e3f0fc747f8cf8b7c633459eddaab1cf6f6812cd846eca0d323294b2c1560f6
=> => exporting attestation manifest sha256:9e2a3ac1957386caa28b8810048ca440dd5a324f5776f7e44b4f64d79614c58f
=> => exporting manifest list sha256:f5311c3e2eae07a02a9dcf921ce9ecfb1e9fb930e888e8e8d779f2d166847d0
=> => naming to docker.io/library/flask-app:1.0
=> => unpacking to docker.io/library/flask-app:1.0
alumno09@A6Alumno09:~/kubernetes-aws-practice/app$ docker images | grep flask-app
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-app:1.0          f5311c3e2eae          197MB          48.1MB
alumno09@A6Alumno09:~/kubernetes-aws-practice/app$

```

Descripción: Proceso de 'docker build' y verificación de la imagen local 'flask-app:1.0'.

```

alumno09@A6Alumno09:~/kubernetes-aws-practice/app$ cd ~/kubernetes-aws-practice/k8s
kubectl delete pods -n flask-lb --all
kubectl get pods -n flask-lb -o wide
pod "flask-app-5480dc4866-2vqg" deleted from flask-lb namespace
pod "flask-app-5480dc4866-c9jm" deleted from flask-lb namespace
pod "flask-app-5480dc4866-nggg" deleted from flask-lb namespace
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
flask-app-5480dc4866-4p855    1/1     Running   0           34s   10.42.0.29     a6alumno09        <none>             <none>
flask-app-5480dc4866-2mkgp    1/1     Running   0           34s   10.42.0.30     a6alumno09        <none>             <none>
flask-app-5480dc4866-1pmq8    1/1     Running   0           34s   10.42.0.28     a6alumno09        <none>             <none>
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$

```

Descripción: Confirmación de los Pods en estado 'Running' tras la importación de la imagen.

## 2.4 Exposición del Servicio (Load Balancer)

Se crea un Service de tipo LoadBalancer. En entornos locales de k3s, esto genera un balanceador que distribuye el tráfico entre los Pods seleccionados.

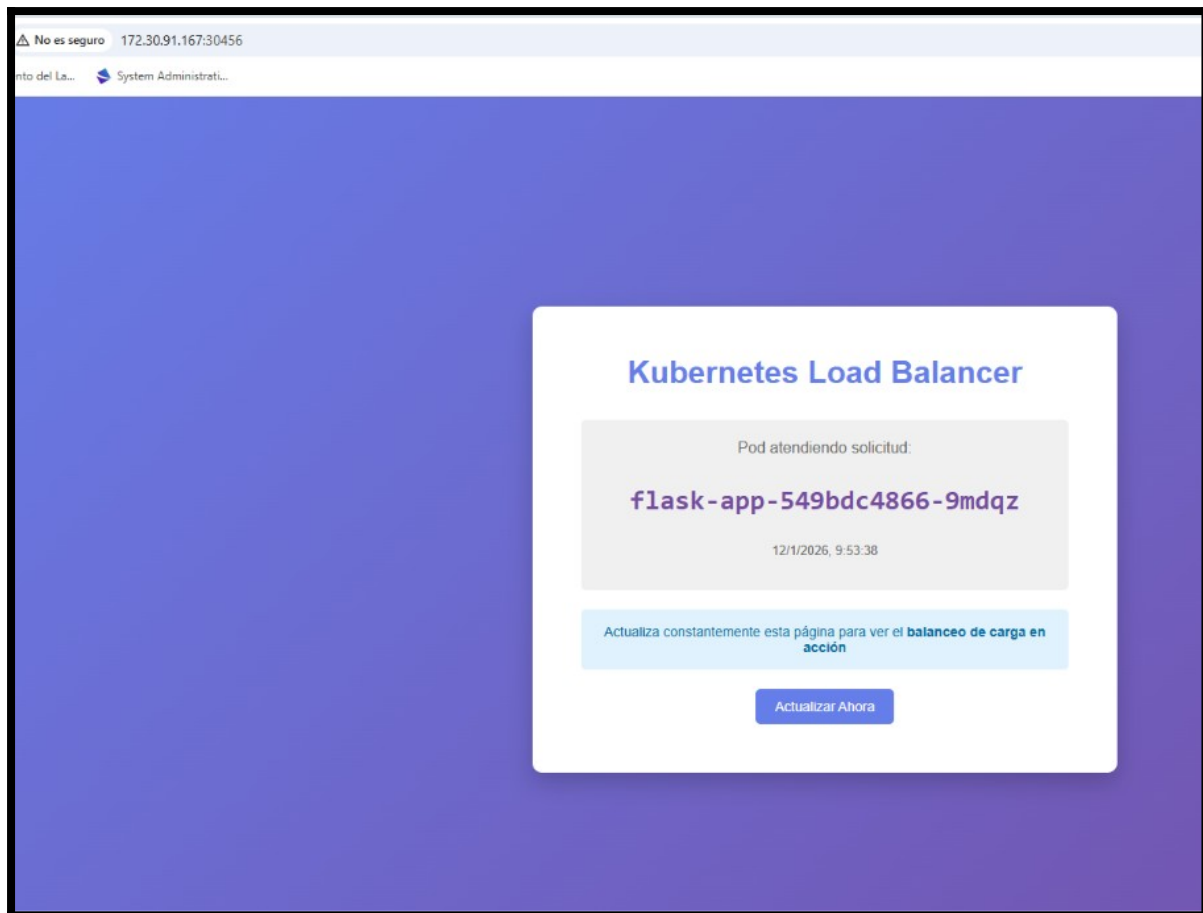
```
alumno09@A6Alumno09: ~/kubernetes-aws-practice/k8s
GNU nano 7.2
apiVersion: v1
kind: Service
metadata:
  name: flask-service
  namespace: flask-lb
spec:
  selector:
    app: flask-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
  type: LoadBalancer
```

Descripción: Definición del Service YAML apuntando al puerto 5000 de los contenedores.

```
Flask-app-549bdc4866-lpmq8 1/1 Running 0 34s 10.42.0.28 a6alumno09 <none> <none>
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ cd ~/kubernetes-aws-practice/k8s
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ nano service.yaml
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ kubectl apply -f service.yaml
service/flask-service created
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ kubectl get svc -n flask-lb -o wide
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE    SELECTOR
flask-service  LoadBalancer  10.43.133.186  <pending>      80:30456/TCP     9s     app=flask-app
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ nano service.yaml
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$ kubectl get svc -n flask-lb -o wide
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE    SELECTOR
flask-service  LoadBalancer  10.43.133.186  <pending>      80:30456/TCP     4m9s   app=flask-app
alumno09@A6Alumno09:~/kubernetes-aws-practice/k8s$
```

Descripción: Aplicación del servicio y visualización de la IP del clúster y puertos asignados.





*Descripción: Acceso vía navegador mostrando el éxito del despliegue y el Pod inicial.*

## PARTE 3: VERIFICACIÓN DEL BALANCEO DE CARGA LOCAL

### 3.1 Acceso mediante Port-Forward

Se utiliza port-forward para mapear el servicio del clúster a un puerto local (8080) y facilitar las pruebas de conectividad iniciales.

```
#6alumno00 Ready control-plane 104m v1.34.2+k3s1 172.30.91.167 <none> Ubuntu 24.04.3 LTS 6.6.87.2-microsoft-standard-WSL2 containerd
#6alumno00@6alumno00:~/kubernetes-aws-practice/k8s$ kubectl port-forward -n flask-lb svc/flask-service 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
```

*Descripción: Comando 'kubectl port-forward' activo redirigiendo tráfico al servicio local.*

## 3.2 Pruebas de Carga Iterativas (Curl)

Se ejecuta un script en bucle para realizar múltiples peticiones y comprobar cómo el Service gestiona el tráfico. Se nota que port-forward tiende a mantener sesiones persistentes, lo cual es un comportamiento esperado para esta herramienta de depuración.

```
C:\Windows\System32>wsl
alumno09@A6Alumno09:/mnt/c/Windows/System32$ for i in {1..10}; do
  echo "Petición $i:"
  curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
  sleep 1
done
Petición 1:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 2:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 3:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 4:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 5:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 6:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 7:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 8:
  "pod_name": "flask-app-549bdc4866-9mdqz",
Petición 9:
  "pod_name": "flask-app-549bdc4866-9mdqz",

```

*Descripción: Resultados del bucle de peticiones curl analizando el nombre de los Pods que responden.*

## PARTE 4: CONFIGURACIÓN EN AWS

### 4.1 Seguridad de Red (Security Groups)

Se crea un grupo de seguridad en AWS permitiendo los protocolos SSH (22), HTTP (80) y HTTPS (443) para garantizar el acceso administrativo y público a la futura instancia.

**Reglas de salida**

| Tipo  | Protocolo | Intervalo de puertos | Destino     | Descripción: opcional | Acción   |
|-------|-----------|----------------------|-------------|-----------------------|----------|
| SSH   | TCP       | 22                   | Anywhere... | SSH                   | Eliminar |
| HTTP  | TCP       | 80                   | Anywhere... | http                  | Eliminar |
| HTTPS | TCP       | 443                  | Anywhere... | https                 | Eliminar |

[Agregar regla](#)

⚠ Las reglas con un destino de 0.0.0.0/0 o ::/0 permiten que las instancias envíen tráfico a cualquier dirección IPv4 o IPv6. Recomendamos configurar las reglas de los grupos de seguridad para que sean más restrictivas y que solo permitan el tráfico a direcciones IP conocidas específicas.

*Descripción: Reglas de entrada configuradas en el Security Group 'kubernetes-aws-sg'.*

## 4.2 Provisión de Instancia EC2

Se lanza una instancia **t3.micro** con Ubuntu 24.04 LTS, asociando el par de claves `kubernetes-key.pem` y asignando una IP pública automática.

**Subred**

Sin preferencias (subred predeterminada en cualquier zona de disponibilidad)

**Asignar automáticamente la IP pública**

Habilitar

**Firewall (grupos de seguridad)**

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☐ Crear grupo de seguridad ☒ Seleccionar un grupo de seguridad existente

**Grupos de seguridad comunes**

Seleccionar grupos de seguridad

kubernetes-aws-sg sg-06bd04363b93444ed

[Compare reglas de grupo de seguridad](#)

**Configurar almacenamiento**

1x 8 GiB gp3 Volumen raíz, 3000 IOPS, No cifrado

[Agregar un nuevo volumen](#)

La AMI seleccionada contiene volúmenes de almacén de instancias; sin embargo, la instancia no permite dichos volúmenes. Por lo que no se podrá obtener acceso a ninguno de estos volúmenes de la AMI desde la instancia.

Haga clic en actualizar para ver la información de la copia de seguridad

Las etiquetas que asigne determinan si alguna política de Data Lifecycle Manager realizará una copia de seguridad de la instancia.

**Resumen**

Número de instancias: 1

**Imagen de software (AMI)**

Canonical, Ubuntu, 24.04, amd64... más información

ami-0ecb62995f68bb549

**Tipo de servidor virtual (tipo de instancia)**

t3.micro

**Firewall (grupo de seguridad)**

kubernetes-aws-sg

**Almacenamiento (volúmenes)**

Volúmenes: 1 (8 GiB)

[Cancelar](#) [Código de](#)

*Descripción: Resumen de configuración de la instancia EC2 antes del lanzamiento.*

## 4.3 Conexión Remota desde WSL2

Se establece una conexión segura mediante SSH utilizando la clave privada descargada, verificando el acceso al sistema operativo en la nube.

```

alumno09@A6Alumno09:/mnt/c/windows/System32$ ssh -i ~/.ssh/kubernetes-key.pem ubuntu@54.145.50.35
The authenticity of host '54.145.50.35 (54.145.50.35)' can't be established.
ED25519 key fingerprint is SHA256:03W6dKH2SwwkvJzW2YGKpbymF435302GV3YsQ1oPmJg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.145.50.35' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Jan 12 11:15:14 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   25.8% of 6.71GB Processes:      111
Memory usage: 23%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.27.41

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-27-41:~$

```

*Descripción: Conexión SSH exitosa desde la terminal de Ubuntu local hacia la IP pública de AWS.*

## 4.4 Instalación de Kubernetes en la Nube

Se prepara la instancia de AWS instalando Docker (para pruebas) y k3s para replicar el entorno de clúster, pero esta vez en una infraestructura cloud real.

```

Alumno09@A6Alumno09: /mnt/c/Windows/System32$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
7eec7bbc9d7: Pull complete
Digest: sha256:d4aaab6242e0cace87e2ec17a2ed3d779d18fbfd03042ea58f2995626396a274
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
Alumno09@A6Alumno09: /mnt/c/Windows/System32$

```

Descripción: Ejecución de 'hello-world' en Docker dentro de la instancia EC2.

```

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Mon Jan 12 11:32:18 2026 from 18.206.107.29
ubuntu@ip-172-31-27-41:~$ curl -sfl https://get.k3s.io | sh -
[INFO] Finding release for channel stable
[INFO] Using v1.34.3+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubect1 symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.service.
[INFO] systemd: Starting k3s
ubuntu@ip-172-31-27-41:~$ █

i-003eaad910d6547ab (kubernetes-node)
PublicIPs: 18.213.4.38 PrivateIPs: 172.31.27.41
CloudShell Comentarios

```

Descripción: Script de instalación de k3s en el servidor remoto de AWS.

```

ubuntu@ip-172-31-27-41:~$ sudo k3s kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
ip-172-31-27-41     Ready    control-plane   41s   v1.34.3+k3s1
ubuntu@ip-172-31-27-41:~$ █

```

Descripción: Verificación del nodo de AWS en estado 'Ready' con rol 'control-plane'.

Se ejecuta un script en la instancia EC2 que solicita información al túnel. El tráfico viaja desde AWS hasta el clúster local de Kubernetes, donde el balanceador decide qué Pod responde.



```
=== Prueba Kubernetes Load Balancer desde AWS ===  
  
Petición 1 → Pod: flask-app-549bdc4866-lpmq8  
Petición 2 → Pod: flask-app-549bdc4866-lpmq8  
Petición 3 → Pod: flask-app-549bdc4866-lpmq8  
Petición 4 → Pod: flask-app-549bdc4866-lpmq8  
Petición 5 → Pod: flask-app-549bdc4866-lpmq8  
Petición 6 → Pod: flask-app-549bdc4866-lpmq8  
Petición 7 → Pod: flask-app-549bdc4866-lpmq8  
Petición 8 → Pod: flask-app-549bdc4866-lpmq8  
Petición 9 → Pod: flask-app-549bdc4866-lpmq8  
Petición 10 → Pod: flask-app-549bdc4866-lpmq8  
Petición 11 → Pod: flask-app-549bdc4866-lpmq8  
Petición 12 → Pod: flask-app-549bdc4866-lpmq8  
Petición 13 → Pod: flask-app-549bdc4866-lpmq8  
Petición 14 → Pod: flask-app-549bdc4866-lpmq8  
Petición 15 → Pod: flask-app-549bdc4866-lpmq8  
  
=== Resumen Balanceo ===  
Pod flask-app-549bdc4866-lpmq8: 15 peticiones  
ubuntu@ip-172-31-74-63:~$ _
```

*Descripción: Ejecución del script 'test-kubernetes-lb.sh' desde AWS y resumen del reparto de carga.*

## PARTE 6: ESCALADO DINÁMICO

### 6.1 Escalado Horizontal de Réplicas

Se demuestra la escalabilidad de Kubernetes aumentando el número de réplicas de 3 a 5 mediante una sola línea de comando. El sistema crea automáticamente dos nuevos Pods para cumplir con el estado deseado.

## PARTE 7: MONITOREO Y OBSERVABILIDAD

### 7.1 Estado de los Recursos

Se verifica el estado general de los Pods y se consultan los logs en tiempo real para observar las peticiones de salud (health checks) realizadas por Kubernetes.



```
alumno09@A6Alumno09: /mnt/c/Windows/System32
Microsoft Windows [Versión 10.0.26100.5074]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>wsl
alumno09@A6Alumno09: /mnt/c/Windows/System32$ sudo k3s kubectl scale deployment flask-app -n flask-lb --replicas=5
[sudo] password for alumno09:
deployment.apps/flask-app scaled
alumno09@A6Alumno09: /mnt/c/Windows/System32$ sudo k3s kubectl get pods -n flask-lb
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-549bdc4866-2wlb6          0/1     Running   0           11s
flask-app-549bdc4866-4p855          1/1     Running   8 (15m ago) 2d3h
flask-app-549bdc4866-9mdqz          1/1     Running   8 (15m ago) 2d3h
flask-app-549bdc4866-lpmq8          1/1     Running   8 (15m ago) 2d3h
flask-app-549bdc4866-pz9b4          0/1     Running   0           11s
alumno09@A6Alumno09: /mnt/c/Windows/System32$
```

Descripción: Listado de los 5 Pods operativos en el Namespace 'flask-lb'.

```
alumno09@A6Alumno09: /mnt/c/Windows/System32
0.42.0.1 - - [14/Jan/2026 12:35:40] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:35:50] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:35:59] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:00] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:10] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:19] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:20] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:30] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:39] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:40] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:50] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:36:59] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:00] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:10] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:19] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:20] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:30] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:39] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:40] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:50] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:37:59] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:00] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:10] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:19] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:20] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:30] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:39] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:40] "GET /health HTTP/1.1" 200 -
0.42.0.1 - - [14/Jan/2026 12:38:50] "GET /health HTTP/1.1" 200 -
alumno09@A6Alumno09: /mnt/c/Windows/System32$
```

Descripción: Logs de un Pod mostrando las respuestas 200 OK a las peticiones del endpoint /health.

## 7.2 Métricas de Rendimiento y Eventos

Se utilizan comandos top para supervisar el consumo de CPU y memoria, asegurando que el clúster no esté saturado. También se analizan los eventos para auditar las acciones recientes del clúster.

```
10.42.0.1 - - [14/Jan/2026 12:36:19] "GET /health HTTP/1.1" 200 -
alumno09@A6Alumno09:/mnt/c/Windows/System32$ sudo k3s kubectl top pods -n flask-lb
sudo k3s kubectl top pods -n flask-lb
NAME                                CPU(cores)   MEMORY(bytes)
flask-app-549bdc4866-2wlbb          1m           21Mi
flask-app-549bdc4866-4p855          1m           21Mi
flask-app-549bdc4866-9mdqz          1m           21Mi
flask-app-549bdc4866-lpmq8          1m           21Mi
flask-app-549bdc4866-pz9b4          1m           21Mi
NAME                                CPU(cores)   MEMORY(bytes)
flask-app-549bdc4866-2wlbb          1m           21Mi
flask-app-549bdc4866-4p855          1m           21Mi
flask-app-549bdc4866-9mdqz          1m           21Mi
flask-app-549bdc4866-lpmq8          1m           21Mi
flask-app-549bdc4866-pz9b4          1m           21Mi
alumno09@A6Alumno09:/mnt/c/Windows/System32$ sudo k3s kubectl top pods -n flask-lb
NAME                                CPU(cores)   MEMORY(bytes)
flask-app-549bdc4866-2wlbb          1m           21Mi
flask-app-549bdc4866-4p855          1m           21Mi
flask-app-549bdc4866-9mdqz          1m           21Mi
flask-app-549bdc4866-lpmq8          1m           21Mi
flask-app-549bdc4866-pz9b4          1m           21Mi
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

Descripción: Consumo de recursos de CPU y Memoria por Pod.

```
alumno09@A6Alumno09:/mnt/c/Windows/System32$ sudo k3s kubectl top nodes
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
alumno09                             113m         0%       1041Mi          13%
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

Descripción: Uso de recursos a nivel de Nodo.

```
alumno09@A6Alumno09:/mnt/c/Windows/System32$ k3s kubectl get events --sort-by='.metadata.creationTimestamp'
LAST SEEN   TYPE      FROM              TO              MESSAGE
16m         Normal    ScalingReplicaSet deployment/flask-app   Scaled up replica set flask-app-549bdc4866 from 3 to 5
28h         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
28h         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
27h         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
27h         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
27h         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
27h         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
23h         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
23h         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
31m         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
31m         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
70m         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
70m         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
52m         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
52m         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
34m         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
34m         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
31m         Normal    EnsuringLoadBalancer service/flask-service Ensuring load balancer
31m         Normal    AppliedDaemonSet  service/flask-service Applied LoadBalancer DaemonSet kube-system
'svc1b-flask-service-d739c1f3
alumno09@A6Alumno09:/mnt/c/Windows/System32$
```

Descripción: Registro de eventos del clúster (escalado, balanceo, etc.).

## 7.3 Detalles del Deployment e Historial

Se inspecciona la configuración profunda del despliegue y se consulta el historial de versiones (rollout history).

```
lumn009@A64lumn009:/mnt/c/windows/System32$ sudo k3s kubectl describe deployment flask-app -n flask-lb
name: flask-app
namespace: flask-lb
creationTimestamp: Mon, 12 Jan 2026 10:00:36 +0100
labels: <none>
annotations: deployment.kubernetes.io/revision: 1
selector: app=flask-app
replicas: 5 desired | 5 updated | 5 total | 5 available | 0 unavailable
strategyType: RollingUpdate
inReadySeconds: 0
rollingUpdateStrategy: 25% max unavailable, 25% max surge
podTemplate:
  labels: app=flask-app
  containers:
    flask-app:
      image: flask-app:1.0
      port: 5000/TCP
      hostPort: 0/TCP
      liveness: http-get http://:5000/health delay=10s timeout=1s period=20s #success=1 #failure=3
      readiness: http-get http://:5000/health delay=5s timeout=1s period=10s #success=1 #failure=3
      environment:
        POD_NAME: (v1:metadata.name)
        POD_NAMESPACE: (v1:metadata.namespace)
      mounts: <none>
      volumes: <none>
      nodeSelectors: <none>
      tolerations: <none>
  conditions:
    type      status      reason
    ----      -
```

```

alumno09@A6Alumno09: /mnt/c/Windows/System32$
12m      Normal      ScalingReplicaSet      deployment/flask-app      Scaled up replica set flask-app-549bdc4866
from 3 to 5
28h      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
28h      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
27h      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
27h      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
27h      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
27h      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
23h      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
23h      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
77m      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
77m      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
66m      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
66m      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
48m      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
48m      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
30m      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
30m      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
27m      Normal      EnsuringLoadBalancer    service/flask-service      Ensuring load balancer
27m      Normal      AppliedDaemonSet         service/flask-service      Applied LoadBalancer DaemonSet kube-system
/svc1b-flask-service-d739c1f3
alumno09@A6Alumno09: /mnt/c/Windows/System32$

```



## PARTE 8: FINALIZACIÓN Y LIMPIEZA

### 8.1 Eliminación de Recursos en Kubernetes

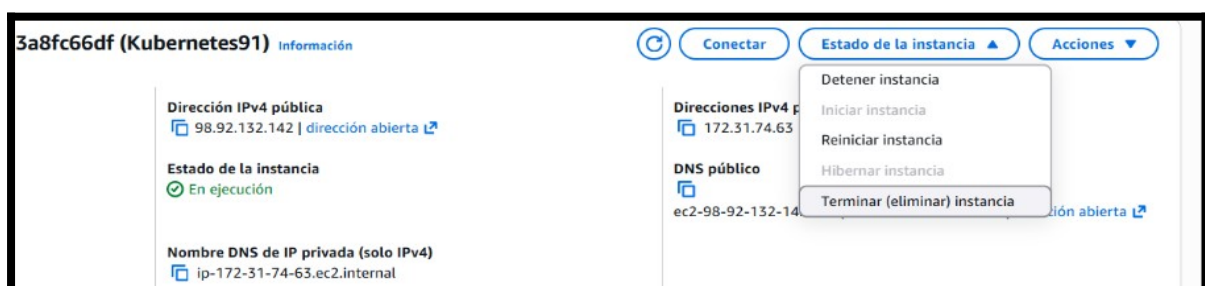
Se elimina el Namespace completo, lo que provoca la eliminación en cascada de todos los servicios, deployments y pods asociados.

```
ilumno09@A6A1umno09: /mnt/c/Windows/System32$ sudo k3s kubectl get namespaces
NAME                STATUS    AGE
default             Active   2d4h
kube-node-lease     Active   2d4h
kube-public         Active   2d4h
kube-system         Active   2d4h
ilumno09@A6A1umno09: /mnt/c/Windows/System32$ sudo k3s kubectl get namespaces
NAME                STATUS    AGE
default             Active   2d4h
kube-node-lease     Active   2d4h
kube-public         Active   2d4h
kube-system         Active   2d4h
ilumno09@A6A1umno09: /mnt/c/Windows/System32$
```

Descripción: Verificación de la limpieza del clúster mediante 'get namespaces'.

### 8.2 Gestión de Infraestructura en AWS

Se procede a terminar (eliminar) la instancia EC2 para evitar costes innecesarios.



Descripción: Estado de la instancia EC2 en la consola de AWS tras solicitar su terminación.

### 8.3 Apagado de los Servicios Locales

Se detiene el servicio k3s en la máquina local para liberar recursos del sistema.

```
alumno09@A6Alumno09:/mnt/c/Windows/System32$ sudo systemctl status k3s
o k3s.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s.service; enabled; preset: enabled)
   Active: inactive (dead) since Wed 2026-01-14 14:00:34 CET; 8s ago
 Duration: 46min 58.537s
    Docs: https://k3s.io
   Process: 171 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, status=0/SUCCESS)
   Process: 180 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Process: 185 ExecStart=/usr/local/bin/k3s server (code=exited, status=0/SUCCESS)
  Main PID: 185 (code=exited, status=0/SUCCESS)
     Tasks: 62
    Memory: 211.9M (peak: 782.9M)
       CPU: 5min 6.134s
    CGroup: /system.slice/k3s.service
            └─2034 /var/lib/rancher/k3s/data/ffd82c56f7aadb3263c7708547a6027705a0eb9a638534465002ba309cae9990/bin/cont>
            └─2174 /var/lib/rancher/k3s/data/ffd82c56f7aadb3263c7708547a6027705a0eb9a638534465002ba309cae9990/bin/cont>
            └─2206 /var/lib/rancher/k3s/data/ffd82c56f7aadb3263c7708547a6027705a0eb9a638534465002ba309cae9990/bin/cont>
            └─2265 /var/lib/rancher/k3s/data/ffd82c56f7aadb3263c7708547a6027705a0eb9a638534465002ba309cae9990/bin/cont>
            └─2609 /var/lib/rancher/k3s/data/ffd82c56f7aadb3263c7708547a6027705a0eb9a638534465002ba309cae9990/bin/cont>

Jan 14 14:00:34 A6Alumno09 k3s[185]: time="2026-01-14T14:00:34+01:00" level=info msg="Closing database connections..."
Jan 14 14:00:34 A6Alumno09 k3s[185]: time="2026-01-14T14:00:34+01:00" level=info msg="TTL events work queue has shut do>
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Deactivated successfully.
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Unit process 2034 (containerd-shim) remains running after unit stop>
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Unit process 2174 (containerd-shim) remains running after unit stop>
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Unit process 2206 (containerd-shim) remains running after unit stop>
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Unit process 2265 (containerd-shim) remains running after unit stop>
Jan 14 14:00:34 A6Alumno09 systemd[1]: k3s.service: Unit process 2609 (containerd-shim) remains running after unit stop>
```

Descripción: Estado 'inactive (dead)' del servicio k3s verificado con `systemctl status`.

## 9. Conclusión y Reflexión Final

### 9.1 Resumen de Logros Técnicos

A lo largo de esta práctica, se ha logrado implementar con éxito un ciclo de vida completo de una aplicación cloud-native:

1. **Desarrollo y Contenerización:** Se transformó una lógica de aplicación Flask en una imagen Docker optimizada.
2. **Orquestación:** Se configuró un clúster k3s en WSL2 capaz de gestionar el estado deseado de la aplicación mediante Deployments y garantizar la alta disponibilidad con réplicas.
3. **Networking Híbrido:** Se superó la barrera de red entre un entorno local (WSL2) y un entorno cloud público (AWS EC2) utilizando túneles SSH, demostrando una solución creativa para pruebas de balanceo de carga sin exponer infraestructura sensible.
4. **Escalabilidad y Salud:** Se validó la capacidad de Kubernetes para autorrepararse (mediante Probes) y escalar dinámicamente según las necesidades del servicio.

## 9.2 Reflexión sobre el Aprendizaje

Esta práctica pone de manifiesto que Kubernetes no es solo una herramienta de despliegue, sino una filosofía de gestión de infraestructura que prioriza la **resiliencia y la automatización**. El uso de **k3s** ha permitido experimentar con la potencia de un orquestador profesional desde un equipo personal, mientras que la integración con **AWS** ha aportado la visión necesaria sobre cómo se gestionan los recursos en proveedores cloud reales.

La principal enseñanza es la versatilidad de la nube híbrida: la capacidad de unir entornos locales y remotos abre un abanico de posibilidades para el desarrollo seguro, el testeo de balanceo de carga y la optimización de costes. En conclusión, el dominio de estos componentes (Contenedores, Orquestación y Conectividad Cloud) es fundamental para cualquier profesional que aspire a diseñar arquitecturas escalables y robustas en la era actual.